Programming Fundamentals

Instructor : Muhammad Owais Idrees

Problem Solving

- Identifying problem and finding the best solution for it.
- Problem solving Techniques:
  - Program
  - Algorithm
  - Flowchart

What is your level of expertise in Programming?

- a) Never programmed before
- b) Have taken at least one course in programming
- c) Have done significant programming
- d) I am a master programmer

The Ideal Way to Do Computing

- The ideal way to ask computer to do something is to order it in a natural language e.g.
  - I want to view this webpage
  - Calculate my annual tax
  - etc.
- However, today's computer's are not intelligent enough to understand our orders in natural language completely.

Where We Are in Computers?

- At the very basic level computers use the concept of an electrical pulse.
  - Low voltage is represented as 0
  - High voltage is represented as 1
- To instruct a computer we need ask the computer in the language of 0s and 1s commonly known as ***machine language***.
- For instance 73 in a number in natural language in the language of 0s and 1s, it becomes 1001001

Programming languages

- Low Level languages
- Assembly languages
- High Level languages

Program Development Process

- Defining and Analyzing Problem
- Designing Algorithm
- Coding
- Testing
- Final Documentation

Scare of Programming?

- Why most students are afraid of programming
  - Paradigm Change
    - Programming is totally different paradigm. You are working on something and you cannot even touch the final output you can only feel it. It is different then other subjects like Physics, Chemistry, Biology, etc.
  - Peer Pressure
    - Some people are naturally good in programming so others think that this is a natural ability and they cannot learn it.

Scare of Programming?

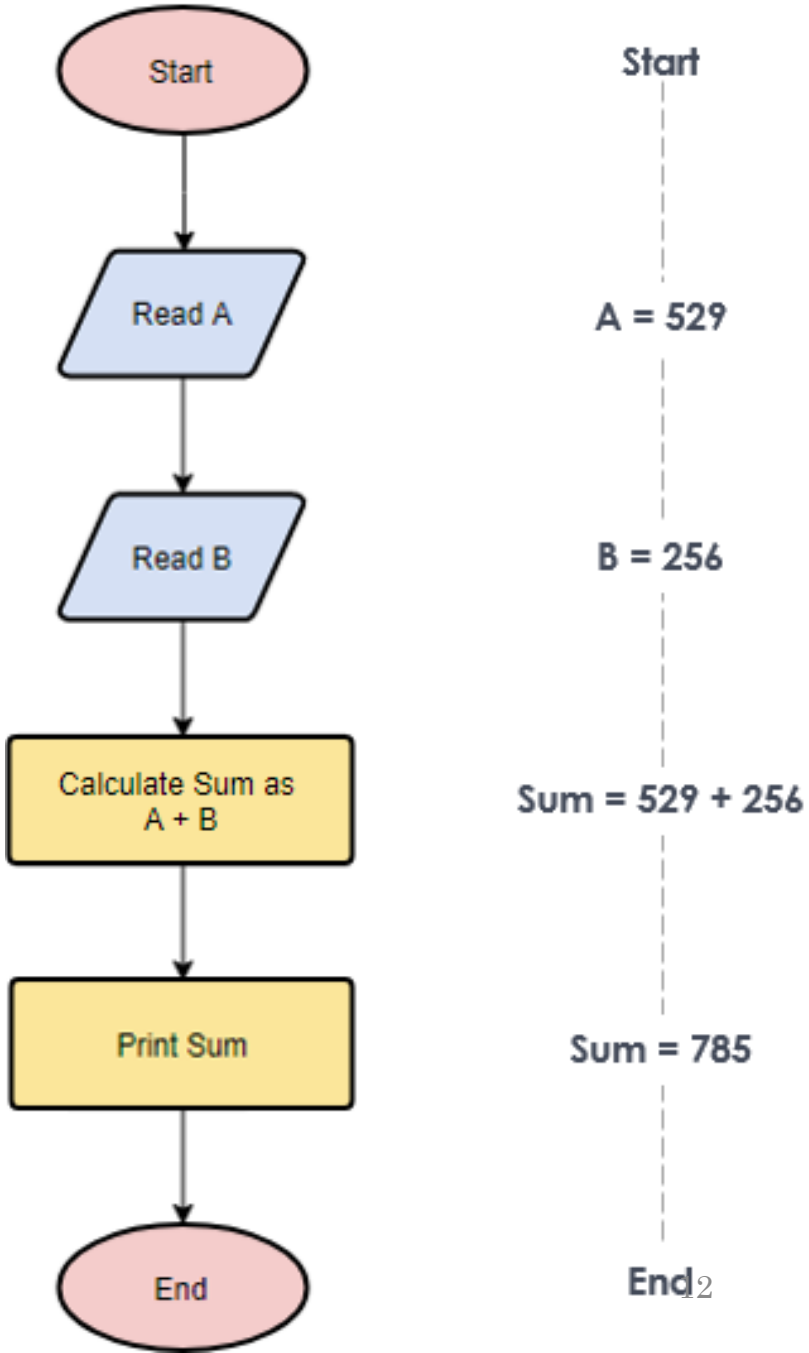
- Lack of Understanding in Fundamental Concepts
  - Some people start programming without a clue of what is going on behind the scene in the computer. As a result they have a flawed understanding from day one of their programming experience
- Time Factor: Programming takes a lot of time
  - Programming may take a lot of time at the start but once a person is comfortable with the concepts and has mastered the basic skills it is just like any other profession.

Flowchart

Symbol	Purpose	Description
	Flow line	Indicates the flow of logic by connecting symbols.
	Terminal(Stop/Start)	Represents the start and the end of a flowchart.
	Input/Output	Used for input and output operation.
	Processing	Used for arithmetic operations and data-manipulations.
	Decision	Used for decision making between two or more alternatives.
	On-page Connector	Used to join different flowline
	Off-page Connector	Used to connect the flowchart portion on a different page.
	Predefined Process/Function	Represents a group of statements performing one processing task.

Example

Find the sum of 529 and 256



Programming Techniques

- Structural Programming
  - Sequential
  - Conditional
  - Iterative
  - Function Call
- Object Oriented Programming
- Event Driven Programming
- Visual Programing

Program Execution

- Source Code
- Compiling
- Linking
- Loading
- Execution

Identifiers

- Standard
- User-define

Data Types

Data Type	Purpose	Range
Int	To store numeric values	2^{-31} to 2^{31}
Float	To store real values	$\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$
Double	To store large real values	2^{-63} to 2^{63}
Char	To store character values	-127 to 127 or 0 to 255

- Overflow
- Underflow

Variable

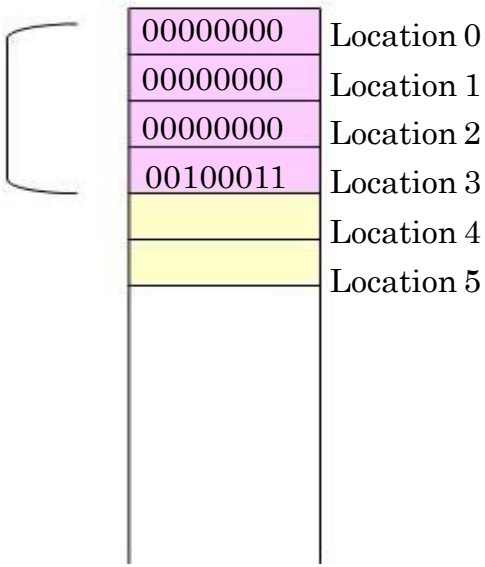
- Declaration
- Initialization

Example of a Variable (Memory View)

int temperature = 35

Locations 0 - 3 are collectively called as 'temperature'

100011 is the binary equivalent of 35

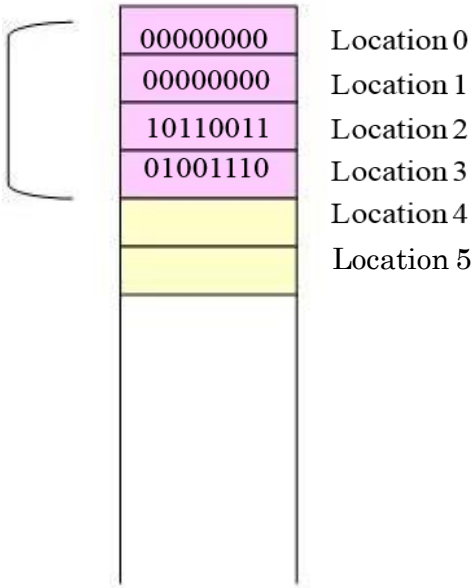


Changing the Value of Variable

- Lets change the value of 'temperature'.

temperature = 45902

Locations 0 - 3 are collectively called as 'temperature'



1011001101001110 is the binary equivalent of 45902

Operators

Type	Operators	Usage
Arithmetic	‘+’ ‘-’ ‘*’ ‘/’ ‘%’	a+b a-b a*b a/b a%b
Arithmetic Assignment	‘+=’ ‘-=’ ‘*=’ ‘/=’ ‘%=’	a+=b is the same as a=a+b a-=b a*=b a/=b a%=b
(Increment and decrement)	‘++’ ‘- -’	a++ is the same as a=a+1 a-- is the same as a=a-1
Relational	‘<’ ‘>’ ‘<=’ ‘>=’ ‘==’ ‘!=’	
Logical	‘&&’ ‘ | | ’, !	

Arithmetic Expression Evaluation

- To evaluate an arithmetic expression two concepts needs to be understood
  - Operator Precedence
    - Operator precedence controls the order in which operations are performed
  - Operator Associativity
    - The associativity of an operator specifies the order in which operations of the same precedence are performed

Arithmetic Operator Precedence and Associativity

Operator(s)	Operation(s)	Order of evaluation (precedence)
( )	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they are evaluated left to right.
* / %	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Precedence of Operators

OPERATORS	TYPE
++ --	unary
* / %	multiplicative
+ -	additive
<> <= >=	relational
== !=	equality
&& | |	logical
= *= /= %= += -=	assignment

Type Casting

- Implicit
  - Performed by Compiler
  - Low to High

- Explicit
  - Performed by programmer

(Type) expression

Some Previous Terms

- Comments
- Uninitialized Variable
- Errors
  - Syntax
  - Logical
  - Runtime
- White Spaces
- Escape Sequence

Input/Output

Input/Output

- Console input
  - cin
- Console Output
  - cout